

Linguagens de Programação

Aplicadas à Análise de Dados



Prof. Dr.

Felipe de Almeida Camargo



Curso

Inteligência e Análise de Dados





Objetivos da aula

- Compreender o papel da linguagem de programação na Ciência de Dados;
- Entender sobre a linguagem **Python** e suas principais aplicações;
- Identificar os principais componentes de um ambiente de desenvolvimento para programação;
- Executar os primeiros programas em Python utilizando o Visual Studio Code;
- Praticar com exercícios.





Motivação

Como empresas transformam dados em decisões?

Imagine uma indústria que registra diariamente:

- Produção de peças;
- Temperatura das máquinas;
- Consumo de energia;
- Quantidade de produtos rejeitados.

Ao final de um mês, milhões de registros foram armazenados.

- **Pergunta:** Como analisar essa quantidade de dados?
- **Resposta:** Utilizando programação.



O que é uma linguagem de programação?

- Uma linguagem de programação é um conjunto de regras formais utilizado para escrever instruções que serão interpretadas ou compiladas e executadas por um computador.
- Ela representa a interface entre o raciocínio humano e a execução computacional.

Na Ciência de Dados, as linguagens permitem:

- Processar grandes volumes de dados;
- Automatizar tarefas repetitivas;
- Construir análises e modelos estatísticos;
- Desenvolver soluções com Inteligência Artificial.





Por que Cientistas de Dados Programam?

- A programação é utilizada para **transformar dados brutos** em **informações** que **apoiam a tomada de decisão**.

Etapas	Exemplo
Coleta	APIs, sensores, arquivos CSV, banco de dados
Limpeza	Corrigir erros, remover duplicidades
Integração	Unir diferentes bases
Análise	Estatística, gráficos, indicadores
Modelagem	<i>Machine Learning</i> e IA
Comunicação	<i>Dashboards</i> e relatórios



Aplicações Reais



Banco

Detectar fraudes em milhões de transações.



Hospital

Prever risco de internação.



Indústria

Prever falhas em equipamentos.



Logística

Otimizar rotas.



Marketing

Recomendar produtos.



Agronegócio

Estimar produtividade.

</> Linguagens mais utilizadas em Ciência de Dados

Linguagem	Principal aplicação	Uso
Python	<i>Machine Learning</i> , Automação, IA	★ ★ ★ ★ ★
SQL	Consulta de Bancos de Dados	★ ★ ★ ★ ★
R	Estatística e Pesquisa Científica	★ ★ ★
Scala	<i>Big Data</i>	★ ★
Julia	Computação Científica	★

Nesta disciplina utilizaremos Python, por ser atualmente a linguagem mais utilizada em Ciência de Dados, Inteligência Artificial e *Machine Learning*.



Por que Python?

Simples de aprender

- Sintaxe clara e objetiva
- Fácil para quem está começando
- Excelente para ensino e prototipagem

Grande ecossistema

- Pandas 
- NumPy 
- Matplotlib 
- Scikit-learn 
- TensorFlow 

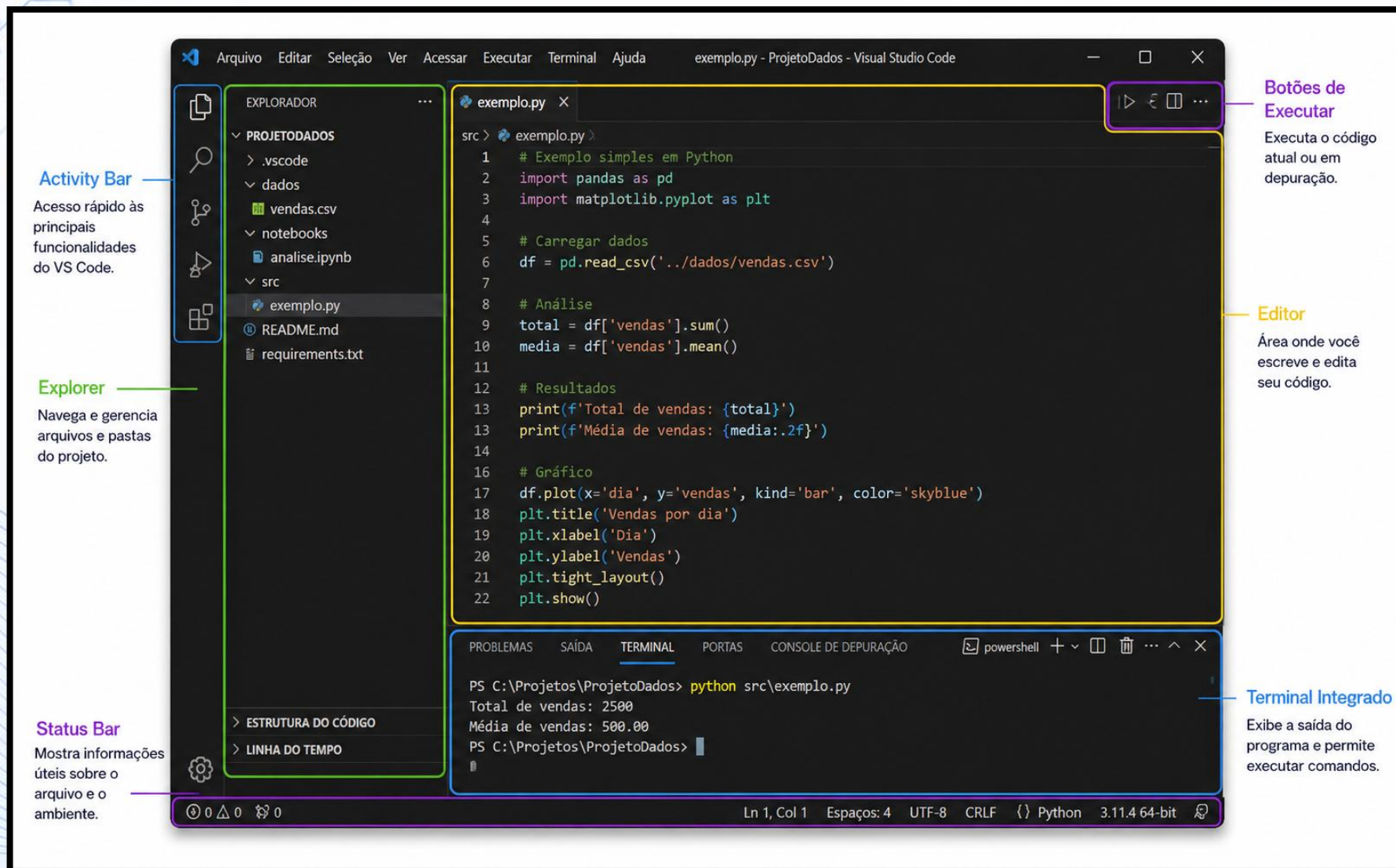
Alta demanda no mercado

- Inteligência Artificial
- *Machine Learning*
- Ciência de Dados
- Automação
- Desenvolvimento Web
- *Big Data*

Python está entre as linguagens mais utilizadas em Ciência de Dados e Inteligência Artificial, sendo adotada por empresas como Google, Microsoft, Netflix, Amazon, Meta e OpenAI.



Ambiente: Visual Studio Code



Download do [Python](#)

Na instalação: marque a opção: “Add Python to PATH”

No VS Code, vá em extensão e instale a extensão do Python.





Funções/ Comandos/ Sintaxe

...





Exibindo informações no Python

- **Situação:** Imagine que você está desenvolvendo o painel eletrônico de recepção da empresa. Sempre que um visitante chega, o sistema deve exibir uma mensagem de boas-vindas.
- Como podemos fazer o computador exibir essa mensagem em Python?

```
1 print("Bem-vindo ao Sistema de Análise de Dados!")
```

- O comando **print()** é uma função utilizada para **exibir informações na tela**.
- Tudo que estiver entre aspas duplas representa um **texto (string)**.

Documentação

Prática: Em Python, exiba um mensagem de boas-vindas ao curso de LPAAD.





Variável e Atribuição de Valores

Uma **variável** é um espaço na memória do computador utilizado para **armazenar informações** que poderão ser utilizadas posteriormente pelo programa. Podemos imaginar uma variável como uma **caixa identificada por um nome**, onde guardamos um valor.

Exemplo:

- a variável **nome** recebe "Felipe"
- a variável **idade** recebe 32
- a variável **profissao** recebe "Professor"

```
1 nome = "Felipe"
2 idade = 32
3 profissao = "Professor"
```

[Documentação](#)

Importante: em programação, o sinal = não significa igualdade matemática. Ele significa "**recebe**" ou "**armazena**". **Obs.:** Dê nomes às variáveis que facilitem a leitura e o entendimento do código.

Prática: Em Python, crie as variáveis **nome**, **idade**, **curso**, **instituição** e **profissão**, atribuindo a elas as suas próprias informações.



Operadores em Python

- Os **operadores** são símbolos utilizados para realizar **operações** com valores e variáveis. Os operadores mais utilizados no início da programação são os **operadores aritméticos**.

Operador	Operação	Exemplo	Resultado
+	Adição	7 + 2	9
-	Subtração	4 - 2	2
*	Multiplicação	3 * 2	6
/	Divisão	5 / 2	2,5
//	Divisão Inteira	5 // 2	2
%	Resto da Divisão	5 % 2	1
**	Potência	3 ** 2	9

- [Documentação](#)



Ordem de Precedência

- Quando uma expressão possui mais de um operador, o Python segue uma ordem para realizar os cálculos.

Prioridade	Operadores
1º	()
2º	**
3º	*, /, //, %
4º	+, -

- Exemplos:**

```
1 resultado = 3 * 5 + 3 ** 2
2 print(resultado)
```

```
1 resultado = 4 - 2**2 + 5/2 + 2*(3+1)
2 print(resultado)
```

Prática: Converta a expressão matemática: $5 \times 2 + 2 \times (2^4 - 4) \div 2$, para a sintaxe do Python, e, em seguida, resolva o mesmo cálculo manualmente para comparar os resultados.

Recebendo Informações do Usuário

- **Situação:** Agora imagine que o sistema precisa saber qual setor da empresa será analisado antes de iniciar o processamento dos dados. Como o computador podemos descobrir o nome do setor que a pessoa trabalha, em Python?

```
1 setor = input("Digite o setor da empresa que você deseja analisar: ")
2 print("Iniciando a análise de dados para o setor:", setor)
```

[Documentação](#)

- **input()** é uma **função** utilizada para **receber informações digitadas pelo usuário**.
- O valor informado é armazenado na variável **setor**. Depois, utilizamos **print()** para exibir essa informação.

Prática: Em Python, solicite ao usuário o seu nome e exiba uma mensagem informando que ele foi cadastrado com sucesso no sistema.



Tipos de Dados

- **O que é um tipo de dado?** Na programação, toda informação armazenada em uma variável possui um **tipo**.
- O tipo determina como aquele valor será armazenado e quais operações podem ser realizadas com ele.

Tipo	Descrição	Exemplo
int	Números Inteiros	0, 1, 2, 3, 4, ...
float	Números Reais	1,1 , 10,5 , 21,4 , ...
str	Texto	"Felipe", "Disciplina", "Bem-vindo", ...



Descobrendo o Tipo de uma Variável

- Até este momento vimos que uma variável pode armazenar diferentes tipos de informações.
 - Mas como podemos descobrir qual é o tipo de uma variável?
 - Para isso utilizamos a função **type()**, que retorna o tipo do valor armazenado.

```
1 idade = 32          # int
2 altura = 1.78       # float
3 nome = "Felipe"     # str
```



```
1 print(type(idade))  # <class 'int'>
2 print(type(altura)) # <class 'float'>
3 print(type(nome))   # <class 'str'>
```

Documentação

Prática: Em Python, solicite ao usuário três informações distintas e utilize a função **type()** para identificar e exibir o tipo de cada variável. O que todas elas têm em comum?

A importância de definir corretamente o tipo da variável

Considere o seguinte programa:

```
1 valor = input("Digite um número: ")
2 print(valor)
3 print(type(valor))
```

Qual será o tipo da variável **valor**?

R: texto (string), mesmo que o usuário digite um número. Para alterar o tipo da variável, basta converter o valor utilizando funções adequadas (**int()** ou **float()**).

```
1 valor = input("Digite um número: ")
2 valor = float(valor)
3 print(type(valor))
```

Por isso, quando desejamos realizar cálculos, precisamos converter esse valor utilizando funções como **int()** ou **float()**.

```
1 valor = float(input("Digite um número: "))
2 print(valor)
3 print(type(valor))
```



Exibindo o Conteúdo de uma Variável

Até agora exibimos **apenas textos escritos** diretamente no programa.

Mas, na prática, normalmente desejamos exibir informações armazenadas em variáveis.

```
1 curso = "Inteligência e Análise de Dados"
2 print(curso)
```

```
1 curso = "Inteligência e Análise de Dados"
2 print("Curso:", curso)
```

- Outra forma muito utilizada para exibir informações é utilizando **f-strings**.
- Ela permite inserir variáveis diretamente dentro de um texto.

```
1 curso = "Inteligência e Análise de Dados"
2 print(f"Curso: {curso}")
```





Caracteres Especiais no print()

Alguns caracteres possuem funções especiais dentro das *strings*.

- **Exemplos:**

Caractere	Descrição
\n	Quebra de linha
\t	Tabulação
\"	Aspas dentro do texto
\\	Barra invertida

```
1 print("Linha 1\nLinha 2")
```

```
1 print("O professor disse: \"Bom trabalho!\")
```

```
1 print("Nome\tNota")
2 print("Felipe\t9.0")
3 print("Bethina\t10.0")
```

```
1 print("C:\\Users\\Felipe")
```

Prática: Em Python, peça ao usuário para fornecer seu salário atual, e retorne o novo salário com 15% de aumento. Exiba uma mensagem ao usuário (usando f-strings) contendo o salário antigo e o novo salário com o aumento.

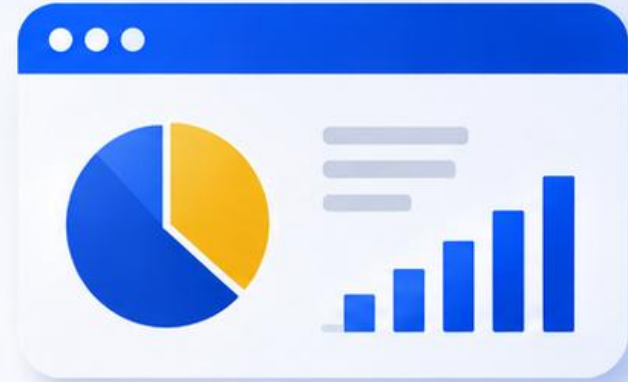


Formatação de números

- Quando trabalhamos com números reais (*float*), muitas vezes o Python exibe diversas casas decimais.
- Podemos controlar essa quantidade utilizando **f-strings**.
- **Exemplo**

```
1  nota = 8.75643
2
3  print(nota)
4  print(f"Nota formatada: {nota:.3f}") # 8.756
5  print(f"Nota formatada: {nota:.2f}") # 8.76
6  print(f"Nota formatada: {nota:.1f}") # 8.8
7  print(f"Nota formatada: {nota:.0f}") # 9
```

Problemas Aplicados





Problema 1: Desenvolvendo uma Calculadora básica

- **Situação:** Você foi selecionado na sua empresa para desenvolver uma **calculadora simples** que será utilizada por outros colaboradores. Ao iniciar o programa, o sistema deve informar ao usuário que ele precisará fornecer dois valores numéricos para que as operações matemáticas sejam realizadas. Após receber os valores, o programa deverá calcular e exibir os resultados básicos das operações aritméticas.

Desenvolva um programa em Python que:

- **Exiba uma mensagem explicando o funcionamento da calculadora.**
- **Solicite ao usuário dois valores numéricos.**
- **Calcule:**
 - Soma, subtração, multiplicação, divisão, divisão inteira, resto da divisão e potência.

Exiba todos os resultados na tela.





Código do Problema 1

```
1 print("=== CALCULADORA SIMPLES ===")
2 print("Este programa realiza as principais operações matemáticas entre dois números.")
3 print("Informe dois números para realizar os cálculos.")
4 print()
5
6 #Entrada
7 valor1 = int(input("Digite o primeiro valor: "))
8 valor2 = int(input("Digite o segundo valor: "))
9
10 #Processamento
11 soma = valor1 + valor2
12 subtracao = valor1 - valor2
13 multiplicacao = valor1 * valor2
14 divisao = valor1 / valor2
15 divisao_inteira = valor1 // valor2
16 resto = valor1 % valor2
17 potencia = valor1 ** valor2
18
19 #Saída
20 print("=== RESULTADOS ===")
21 print("Soma:", soma)
22 print("Diferença:", subtracao)
23 print("Produto:", multiplicacao)
24 print("Quociente:", divisao)
25 print("Divisão inteira:", divisao_inteira)
26 print("Resto da divisão:", resto)
27 print("Potência:", potencia)
```

Desafio: Como você faria a raiz quadrada de um número? Sem usar comandos prontos?





Problema 2: Controle de Produção

- **Situação:** Uma indústria registrou a quantidade de peças produzidas durante cinco dias da semana. O gerente deseja desenvolver um programa para analisar esses dados e gerar um pequeno relatório da produção semanal.

Dia	Produção
Segunda	1.240
Terça	1.315
Quarta	1.288
Quinta	1.360
Sexta	1.297

Desenvolva um programa em Python que:

- Armazene a produção de cada dia em uma variável.
- Calcule a **produção total** da semana.
- Calcule a produção **média diária**.
- Determine quantas peças faltariam para atingir a meta semanal de **7.000 peças**.
- Exiba todas as informações na tela.





Código do Problema 2

```
1 print("=== CONTROLE DE PRODUÇÃO ===")
2 print("Este programa calcula informações sobre a produção semanal.")
3
4 # Entrada
5 segunda = 1240
6 terca = 1315
7 quarta = 1288
8 quinta = 1360
9 sexta = 1297
10
11 # Processamento
12 total = segunda + terca + quarta + quinta + sexta
13 media = total / 5
14
15 meta = 7000
16 diferenca = meta - total
17
18 # Saída
19 print("=== RELATÓRIO DE PRODUÇÃO ===")
20 print(f"Produção total da semana: {total} peças")
21 print(f"Produção média diária: {media} peças")
22 print(f"Meta semanal: {meta} peças")
23 print(f"Diferença para a meta: {diferenca} peças")
```





Problema 3: Análise de Notas de um Aluno

- **Situação:** Uma escola deseja analisar o desempenho de um aluno ao longo do semestre. Para isso, será desenvolvido um programa em Python capaz de receber as informações do aluno e suas cinco notas, gerando indicadores que auxiliem o professor na avaliação do desempenho acadêmico.

Desenvolva um programa em Python que:

- Solicite ao aluno, o **nome**, a **disciplina**, **professor** e as **cinco notas**;
- Calcule a **média das notas**;
- **Exiba uma mensagem personalizada utilizando uma f-string**, contendo o nome do aluno, a disciplina, o professor e a média final, apresentando os valores numéricos com **duas casas decimais**.



Código Problema 3

```
1 print("==== Análise de Desempenho do Aluno ====")
2
3 # Entrada
4 nome = input("Digite o nome do aluno: ")
5 disciplina = input("Digite a disciplina: ")
6 professor = input("Digite o nome do professor: ")
7
8 nota1 = float(input("Digite a primeira nota: "))
9 nota2 = float(input("Digite a segunda nota: "))
10 nota3 = float(input("Digite a terceira nota: "))
11 nota4 = float(input("Digite a quarta nota: "))
12 nota5 = float(input("Digite a quinta nota: "))
13
14 # Processamento
15 media = (nota1 + nota2 + nota3 + nota4 + nota5) / 5
16
17 # Saída
18 print("==== RELATÓRIO DE DESEMPENHO ====")
19 print(f"Aluno(a): {nome}, matriculado na disciplina: {disciplina}, ministrada pelo(a) Professor(a): {professor}, obteve uma média final de: {media:.2f}")
```

Problema 4: Monitoramento de Temperatura Industrial

- **Situação:** Uma máquina de uma indústria possui sensores que registram automaticamente sua temperatura ao longo do expediente. O engenheiro de manutenção deseja analisar os dados coletados para verificar se a máquina está operando dentro do comportamento esperado.

Desenvolva um programa em Python que calcule:

- A **temperatura média** da máquina.
- Exiba manualmente a **temperatura máxima e mínima** registrada.
- Calcule a **amplitude térmica** (máxima – mínima).

Considerando que a temperatura ideal de operação é **73 °C**,

- Calcule a **diferença** entre a temperatura média e a temperatura ideal;
- Exiba uma mensagem utilizando uma f-string, informando a temperatura média da máquina, a amplitude térmica e a diferença em relação à temperatura ideal, apresentando os valores com uma casa decimal.

Horário	Temperatura (°C)
08:00	72,3
09:00	73,1
10:00	74,0
11:00	72,8
12:00	71,5
13:00	75,2
14:00	73,6
15:00	74,4



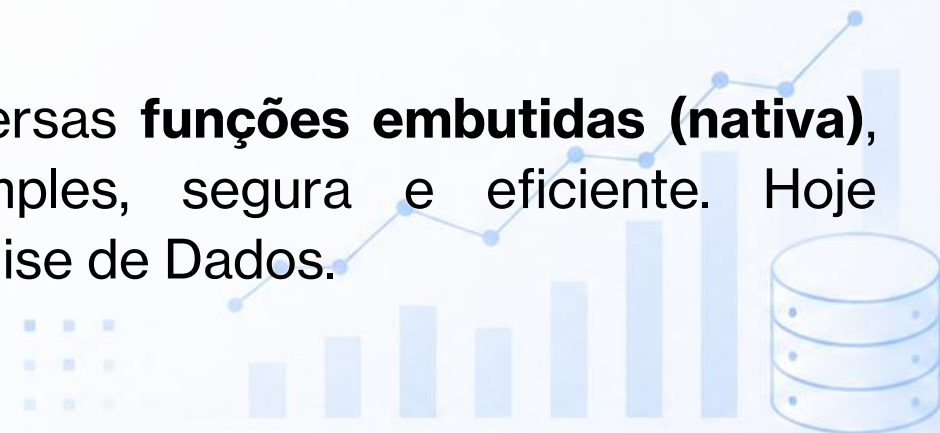
Código Problema 4

```
1  print("==== MONITORAMENTO DE TEMPERATURA INDUSTRIAL ====")
2  print()
3
4  # Temperaturas registradas
5  t1 = 72.3
6  t2 = 73.1
7  t3 = 74.0
8  t4 = 72.8
9  t5 = 71.5
10 t6 = 75.2
11 t7 = 73.6
12 t8 = 74.4
13
14 # Soma das temperaturas
15 soma = t1 + t2 + t3 + t4 + t5 + t6 + t7 + t8
16
17 # Temperatura média
18 media = soma / 8
19
20 # Temperatura máxima (identificada manualmente)
21 maior = t6
22
23 # Temperatura mínima (identificada manualmente)
24 menor = t5
25
26 # Amplitude térmica
27 amplitude = maior - menor
28
29 # Temperatura ideal
30 temperatura_ideal = 73.0
31
32 # Diferença entre a média e a temperatura ideal
33 diferenca = abs(media - temperatura_ideal)
34
35 # Exibição dos resultados
36 print(f"Temperatura média: {media:.1f} °C")
37 print(f"Temperatura máxima: {maior:.1f} °C")
38 print(f"Temperatura mínima: {menor:.1f} °C")
39 print(f"Amplitude térmica: {amplitude:.1f} °C")
40 print()
41
42 print(
43     f"A temperatura média da máquina foi de {media:.1f} °C, "
44     f"a amplitude térmica foi de {amplitude:.1f} °C "
45     f"e a diferença em relação à temperatura ideal "
46     f"{(temperatura_ideal:.1f} °C) foi de {diferenca:.1f} °C."
47 )
```



Da lógica à otimização

- Até este momento, realizamos todos os cálculos "passo a passo". Essa abordagem é importante porque nos ajuda a compreender a lógica por trás de cada operação realizada pelo computador.
- Entretanto, à medida que os problemas se tornam maiores e mais complexos, escrever todas as operações manualmente torna-se trabalhoso, aumenta a chance de erros e dificulta a manutenção do código.
- Para facilitar esse trabalho, o Python disponibiliza diversas **funções embutidas (nativa)**, que executam operações comuns de forma simples, segura e eficiente. Hoje conheceremos três funções bastante utilizadas em Análise de Dados.





Funções

- **Função sum()**
- A função sum() retorna a **soma de todos os elementos** de uma sequência de valores.
- Utilizada para calcular totais, somatórios e acumulados.
- **Exemplo**

```
1 total = sum((72.3, 73.1, 74.0, 72.8, 71.5, 75.2, 73.6, 74.4))
2 print(total)
```

[Documentação](#)

- **Função max()**
- A função **max()** retorna o **maior valor** entre os valores informados.
- **Exemplo**

```
1 maior = max(72.3, 73.1, 74.0, 72.8, 71.5, 75.2, 73.6, 74.4)
2 print(maior)
```

[Documentação](#)

- **Função min()**
- A função min() retorna o **menor valor** entre os dados informados.
- **Exemplo**

```
1 menor = min(72.3, 73.1, 74.0, 72.8, 71.5, 75.2, 73.6, 74.4)
2 print(menor)
```

[Documentação](#)





Otimizando o código

```
1 soma = t1 + t2 + t3 + t4 + t5 + t6 + t7 + t8
```



```
1 total = sum((72.3, 73.1, 74.0, 72.8, 71.5, 75.2, 73.6, 74.4))  
2 print(total)
```

Definimos manualmente quem era o maior

```
1 # Maior temperatura (identificada manualmente)  
2 maior = t6
```



```
1 maior = max(72.3, 73.1, 74.0, 72.8, 71.5, 75.2, 73.6, 74.4)  
2 print(maior)
```

Definimos manualmente quem era o menor

```
1 # Menor temperatura (identificada manualmente)  
2 menor = t5
```



```
1 menor = min(72.3, 73.1, 74.0, 72.8, 71.5, 75.2, 73.6, 74.4)  
2 print(menor)
```



Código Problema 4 Otimizado

```
1  print("==== MONITORAMENTO DE TEMPERATURA INDUSTRIAL ====")
2  print()
3
4  # Temperaturas registradas
5  t1 = 72.3
6  t2 = 73.1
7  t3 = 74.0
8  t4 = 72.8
9  t5 = 71.5
10 t6 = 75.2
11 t7 = 73.6
12 t8 = 74.4
13
14 # Temperatura ideal
15 temperatura_ideal = 73.0
16
17 # Cálculos utilizando funções do Python
18 media = sum((t1, t2, t3, t4, t5, t6, t7, t8)) / 8
19 maior = max(t1, t2, t3, t4, t5, t6, t7, t8)
20 menor = min(t1, t2, t3, t4, t5, t6, t7, t8)
21
22 # Amplitude térmica
23 amplitude = maior - menor
24
25 # Diferença em relação à temperatura ideal
26 diferenca = abs(media - temperatura_ideal)
27
28 # Exibição dos resultados
29 print(f"Temperatura média: {media:.1f} °C")
30 print(f"Temperatura máxima: {maior:.1f} °C")
31 print(f"Temperatura mínima: {menor:.1f} °C")
32 print(f"Amplitude térmica: {amplitude:.1f} °C")
33 print()
34
35 print(
36     f"A temperatura média da máquina foi de {media:.1f} °C, "
37     f"a amplitude térmica foi de {amplitude:.1f} °C "
38     f"e a diferença em relação à temperatura ideal "
39     f"({temperatura_ideal:.1f} °C) foi de {diferenca:.1f} °C."
40 )
```



Problema 5: Controle de Qualidade

- **Situação:** Uma indústria possui **quatro linhas de produção** e registrou a quantidade de peças produzidas durante uma semana.

Linha	Segunda	Terça	Quarta	Quinta	Sexta
A	82	91	88	95	90
B	96	93	98	89	94
C	74	79	76	81	77
D	85	83	87	90	88

Responda:

- Calcule a **produção total** de cada linha.
- Calcule a **produção total da fábrica** durante a semana.
- Calcule a **produção média** de cada linha.
- Calcule a **produção média diária** da fábrica.
- Identifique a **linha mais produtiva** e a **menos produtiva**.
- Calcule a diferença de produção entre essas duas linhas (mais e menos produtiva).





Código do Problema 5

```
1 # Linha A
2 total_a = sum((82, 91, 88, 95, 90))
3 # Linha B
4 total_b = sum((96, 93, 98, 89, 94))
5 # Linha C
6 total_c = sum((74, 79, 76, 81, 77))
7 # Linha D
8 total_d = sum((85, 83, 87, 90, 88))
9
10 print("Produção Linha A:", total_a)
11 print("Produção Linha B:", total_b)
12 print("Produção Linha C:", total_c)
13 print("Produção Linha D:", total_d)
14
15 # Produção total da fábrica
16 total_fabrica = sum((total_a, total_b, total_c, total_d))
17 print("\nProdução total da fábrica:", total_fabrica)
18
19 # Média de cada linha
20 media_a = total_a / 5
21 media_b = total_b / 5
22 media_c = total_c / 5
23 media_d = total_d / 5
24
25 print("\nMédia Linha A:", media_a)
26 print("Média Linha B:", media_b)
27 print("Média Linha C:", media_c)
28 print("Média Linha D:", media_d)
29
30 # Média da fábrica
31 media_fabrica = total_fabrica / 20
32 print("\nMédia diária da fábrica:", media_fabrica)
33
34 # Maior e menor produção
35 maior = max(total_a, total_b, total_c, total_d)
36 menor = min(total_a, total_b, total_c, total_d)
37 print("\nMaior produção:", maior)
38 print("Menor produção:", menor)
39
40
41 # Diferença
42 diferenca = maior - menor
43 print("\nDiferença:", diferenca)
```

Problema 6: Indicadores de Qualidade

Situação: Uma indústria monitora mensalmente a quantidade de peças produzidas e o número de peças defeituosas como forma de avaliar a qualidade do processo produtivo. A partir dessas informações, o setor de Engenharia da Qualidade calcula dois indicadores para cada mês: taxa de rejeição e o percentual de aprovação.

$$\bullet TR = \frac{\text{Peças Defeituosas}}{\text{Produção Mensal}} \times 100\%$$

Desenvolva um programa em Python que:

- Armazene as informações em variáveis.
- Calcule a **Taxa de Rejeição (%)** de cada mês.
- Calcule o **Percentual de Peças Aprovadas (%)** de cada mês.
- A **média** da Taxa de Rejeição e do percentual de peças aprovadas.
- Exiba um relatório contendo todas as informações obtidas.

Mês	Produção	Peças com defeito
Jan	1.140	25
Fev	1210	22
Mar	1250	16
Abr	1160	31
Mai	1207	19
Jun	1205	25

Código do Problema 6

```
1 print("===== INDICADORES DE QUALIDADE =====")
2 print()
3
4 # Dados de produção e peças defeituosas
5 producao_jan = 1140
6 defeitos_jan = 25
7
8 producao_fev = 1210
9 defeitos_fev = 22
10
11 producao_mar = 1250
12 defeitos_mar = 16
13
14 producao_abr = 1160
15 defeitos_abr = 31
16
17 producao_mai = 1207
18 defeitos_mai = 19
19
20 producao_jun = 1205
21 defeitos_jun = 25
22
23 # Taxa de Rejeição
24 taxa_rejeicao_jan = (defeitos_jan / producao_jan) * 100
25 taxa_rejeicao_fev = (defeitos_fev / producao_fev) * 100
26 taxa_rejeicao_mar = (defeitos_mar / producao_mar) * 100
27 taxa_rejeicao_abr = (defeitos_abr / producao_abr) * 100
28 taxa_rejeicao_mai = (defeitos_mai / producao_mai) * 100
29 taxa_rejeicao_jun = (defeitos_jun / producao_jun) * 100
30
31 # Percentual de Aprovação
32 aprovacao_jan = 100 - taxa_rejeicao_jan
33 aprovacao_fev = 100 - taxa_rejeicao_fev
34 aprovacao_mar = 100 - taxa_rejeicao_mar
35 aprovacao_abr = 100 - taxa_rejeicao_abr
36 aprovacao_mai = 100 - taxa_rejeicao_mai
37 aprovacao_jun = 100 - taxa_rejeicao_jun
38
39 # Produção total
40 producao_total = sum((producao_jan, producao_fev, producao_mar, producao_abr, producao_mai, producao_jun))
41
42 # Total de peças com defeito
43 defeitos_total = sum((defeitos_jan, defeitos_fev, defeitos_mar, defeitos_abr, defeitos_mai, defeitos_jun))
44
45 # Média da Taxa de Rejeição
46 media_rejeicao = sum((taxa_rejeicao_jan, taxa_rejeicao_fev, taxa_rejeicao_mar, taxa_rejeicao_abr, taxa_rejeicao_mai, taxa_rejeicao_jun)) / 6
47
48 # Média do Percentual de Aprovação
49 media_aprovacao = sum((aprovacao_jan, aprovacao_fev, aprovacao_mar, aprovacao_abr, aprovacao_mai, aprovacao_jun)) / 6
50
51 # Maior e menor Taxa de Rejeição
52 maior_rejeicao = max(taxa_rejeicao_jan, taxa_rejeicao_fev, taxa_rejeicao_mar, taxa_rejeicao_abr, taxa_rejeicao_mai, taxa_rejeicao_jun)
53
54 menor_rejeicao = min(taxa_rejeicao_jan, taxa_rejeicao_fev, taxa_rejeicao_mar, taxa_rejeicao_abr, taxa_rejeicao_mai, taxa_rejeicao_jun)
55
```

```
56 # Relatório mensal
57 print("----- JANEIRO -----")
58 print(f"Produção: {producao_jan} peças")
59 print(f"Peças com defeito: {defeitos_jan}")
60 print(f"Taxa de Rejeição: {taxa_rejeicao_jan:.2f}%")
61 print(f"Percentual de Aprovação: {aprovacao_jan:.2f}%")
62 print()
63
64 print("----- FEVEREIRO -----")
65 print(f"Produção: {producao_fev} peças")
66 print(f"Peças com defeito: {defeitos_fev}")
67 print(f"Taxa de Rejeição: {taxa_rejeicao_fev:.2f}%")
68 print(f"Percentual de Aprovação: {aprovacao_fev:.2f}%")
69 print()
70
71 print("----- MARÇO -----")
72 print(f"Produção: {producao_mar} peças")
73 print(f"Peças com defeito: {defeitos_mar}")
74 print(f"Taxa de Rejeição: {taxa_rejeicao_mar:.2f}%")
75 print(f"Percentual de Aprovação: {aprovacao_mar:.2f}%")
76 print()
77
78 print("----- ABRIL -----")
79 print(f"Produção: {producao_abr} peças")
80 print(f"Peças com defeito: {defeitos_abr}")
81 print(f"Taxa de Rejeição: {taxa_rejeicao_abr:.2f}%")
82 print(f"Percentual de Aprovação: {aprovacao_abr:.2f}%")
83 print()
84
85 print("----- MAIO -----")
86 print(f"Produção: {producao_mai} peças")
87 print(f"Peças com defeito: {defeitos_mai}")
88 print(f"Taxa de Rejeição: {taxa_rejeicao_mai:.2f}%")
89 print(f"Percentual de Aprovação: {aprovacao_mai:.2f}%")
90 print()
91
92 print("----- JUNHO -----")
93 print(f"Produção: {producao_jun} peças")
94 print(f"Peças com defeito: {defeitos_jun}")
95 print(f"Taxa de Rejeição: {taxa_rejeicao_jun:.2f}%")
96 print(f"Percentual de Aprovação: {aprovacao_jun:.2f}%")
97 print()
98
99 # Relatório final
100 print("===== RESUMO DO PERÍODO =====")
101 print(f"Produção total: {producao_total} peças")
102 print(f"Total de peças com defeito: {defeitos_total}")
103 print(f"Média da Taxa de Rejeição: {media_rejeicao:.2f}%")
104 print(f"Média do Percentual de Aprovação: {media_aprovacao:.2f}%")
105 print(f"Maior Taxa de Rejeição: {maior_rejeicao:.2f}%")
106 print(f"Menor Taxa de Rejeição: {menor_rejeicao:.2f}%")
```

Atividades

1	Exercício 1 – Primeiro Programa	4
2	Exercício 2 – Variáveis	4
3	Exercício 3 – Recebendo Informações do Usuário	4
4	Exercício 4 – Descobrindo o Tipo das Variáveis	4
5	Exercício 5 – Convertendo Tipos	5
6	Exercício 6 – Operadores Aritméticos	5
7	Exercício 7 – Formatação com f-strings	5
8	Exercício 8 – Ordem de Precedência	6
9	Exercício 9 – Conversão de Temperatura	6
10	Exercício 10 – Índice de Massa Corporal	6
11	Exercício 11 – Velocidade Média	7
12	Exercício 12 – Consumo Médio de Combustível	7
13	Exercício 13 – Utilizando a função <code>sum()</code>	8
14	Exercício 14 – Utilizando a função <code>max()</code>	8
15	Exercício 15 – Utilizando as funções <code>sum()</code> , <code>max()</code> e <code>min()</code>	8
16	Exercício 16 – Controle de Estoque	9
17	Exercício 17 – Receita de Vendas	9
18	Desafio: Índice de Eficiência Operacional	10



Versionamento

- No Github crie um repositório LPAAD_Aulas (deixe Público e sem o README).
- No notebook/PC crie uma pasta acesse essa pasta pelo VS Code;
- Acesso o terminal do VS Code e execute:
- `git init;`
- `git config --global user.name "Seu nome";`
- `git config --global user.email "Seu email";`
- `git add .` (Crie um README.md)
- `git commit --m "Aula 1, práticas e problemas"`
- `git branch --M main` (Para garantir estar na *main*)
- `git remote add origin url_do_repositório;`
- `git push --u origin main;`



Obrigado!

